32-bit CPU
Programmer User Manual

Assembly editor, behavioral simulation, FPGA upload, and live register monitoring

CPU design	32-bit, five-stage pipeline: IF / ID / EX / MEM / WB
FPGA target	Digilent Nexys A7-100T, Artix-7 `xc7a100tcsg324-1`
Clock / reset	100 MHz clock; active-low external reset input
Programming	Python/Tkinter GUI sends assembled 32-bit words over UART
Monitor	Program counter, R0-R31, and halt status shown in the GUI

Quick start

1. Install Python 3. For FPGA mode, install the serial dependency with `pip install pyserial`.

2. Open a terminal in the folder containing `assembler.py` and run `python cpu_assembler_gui_fresh.py`.

3. Write or open an assembly program, save it as a `.asm` file, and include HLT at the end.

4. For simulation, choose Simulation and click Build / Sim. For hardware, choose Nexys A7, connect to the correct COM port at 921600 baud, then click Send to FPGA.

Important: simulation paths are machine-specific. The supplied batch, Tcl, and testbench use `D:/assembler` and the batch file expects Vivado under `D:/2025.1/Vivado/bin`. Keep that layout or update those paths before running on a different computer.

Files to know

File	Why it matters
cpu_assembler_gui_fresh.py	Main graphical editor, assembler controls, simulation launcher, UART monitor.
assembler.py	Assembly-to-machine-code logic. Keep it beside the GUI script.
cpu_32bit.xpr	Vivado project. Hardware top module: nexys_a7_top.
cpu_sim_tb.v	Simulation testbench; sends program words through the simulated UART and writes JSON.
nexys_a7_top.xdc	Clock, UART, and reset pin constraints.

1. Writing an assembly program

Use register names R0 through R31. Instruction mnemonics are case-insensitive. A line beginning with # is a comment. Labels end with a colon and may be used by branches and jumps.

Instruction reference

Group	Instruction format	Operation
Load immediate	`MOVI Rt, imm`	Load a 16-bit immediate; zero-extend it.
Arithmetic	`ADD Rd, Rs, Rt`	Add two register values.
	`SUB Rd, Rs, Rt`	Subtract Rt from Rs.
	`MUL Rd, Rs, Rt`	Multiply two register values; write 32-bit result.
	`ADDI Rt, Rs, imm`	Add sign-extended immediate to Rs.
	`SUBI Rt, Rs, imm`	Subtract sign-extended immediate from Rs.
Logic / compare	`AND Rd, Rs, Rt`	Bitwise AND.
	`OR Rd, Rs, Rt`	Bitwise OR.
	`SLT Rd, Rs, Rt`	Set Rd = 1 when signed Rs < Rt; otherwise 0.
	`SLTI Rt, Rs, imm`	Set Rt = 1 when signed Rs < immediate; otherwise 0.
Move	`MOV Rd, Rs`	Copy Rs to Rd.
Memory	`LW Rt, off(Rs)`	Load a 32-bit word from address Rs + offset.
	`SW Rt, off(Rs)`	Store a 32-bit word at address Rs + offset.
Control flow	`BEQZ Rs, label`	Branch when Rs is zero.
	`BNEQZ Rs, label`	Branch when Rs is nonzero.
	`J label`	Jump to a label.
Stop	`HLT`	Halt execution.

Example program

```
# Count R1 down from 5 to zero.
MOVI R1, 5
MOVI R2, 1

LOOP:
SUBI R1, R1, 1
BNEQZ R1, LOOP

MOV R3, R1
HLT
```

The assembler checks register names, operand counts, label definitions, and the 16-bit immediate range. Memory operands use `offset(Rs)`. Use aligned word addresses for LW/SW (for example 0, 4, 8).

Machine code and the loader marker

The assembler displays 32-bit words and writes them to `program.hex`. It appends `0xFFFFFFFF` as a loader end marker. Do not put that marker in your source code. It is separate from source-level HLT, encoded as `0xFC000000`.

2. Run a behavioral simulation

Simulation uses Vivado XSim. The testbench loads the generated hex words through the same UART receiver and program-loader path used on the board, waits for HLT or a timeout, and writes the CPU state to JSON.

Steps

- Save your source. In the GUI, open or enter a program and save it as a .asm file. Keep the file in the intended working directory.
- Select Simulation. The Simulation mode disables COM-port controls and enables Build / Sim.
- Check Settings. Under Tools / Simulation Settings, check the simulation command (default `run_simulation_fresh.bat`) and JSON filename (default `simulation_result.json`).
- Run. Click Build / Sim or press F7. The GUI assembles the code, writes `program.hex`, launches the script, and reads the generated JSON.
- Inspect the result. Look at PC, R0-R31, halt status, and the simulation status in the GUI. A normal completed program should reach HLT.

Expected outputs

File / field	Meaning
program.hex	One 8-digit hexadecimal word per line. Generated by the GUI beside the current assembly file.
simulation_result.json	Machine-readable final CPU state written by cpu_sim_tb.v.
status	HALTED, TIMEOUT, or an input/load error status.
registers	Array of 32 hex strings, R0 through R31.
pc / halt	Final program counter and halt flag. Additional fields include cycle count and unknown-register count.

Path setup is the main portability issue. The supplied .bat, .tcl, and testbench contain fixed Windows paths. In the batch file, update the Vivado installation location and project location. In the Tcl and testbench, update the project, program.hex, and JSON output paths to match. Make the GUI result filename and output location agree with the testbench.

Relevant path settings in the supplied project

Location	Current expected path
Batch file Vivado folder	`D:\2025.1\Vivado\bin`
Vivado project	`D:/assembler/cpu_32bit.xpr`
Hex input	`D:/assembler/program.hex`
JSON output	`D:/assembler/simulation_result.json`

The archived `simulation_result.json` is only an example snapshot. It records a halted run with zero reported errors and zero unknown registers; your own result will depend on your program.

3. Program the Nexys A7

First build and program the FPGA design. The hardware top module is `nexys_a7_top`; the CPU and UART interfaces are connected in `cpu_top.v`.

Build the bitstream in Vivado

- Open `cpu_32bit.xpr` in Vivado 2025.1.
- Verify `nexys_a7_top` is selected as the synthesis top and `nexys_a7_top.xdc` is enabled.
- Run Synthesis, then Implementation, then Generate Bitstream.
- Open Hardware Manager, connect to the FPGA, and program the generated .bit file.

Pins and serial settings

Signal	Pin	Purpose
clk	E3	100 MHz input clock (10 ns period).
uart_rx_pin	C4	Host to FPGA; program upload bytes.
uart_tx_pin	D4	FPGA to host; debug packets.
resetn	C12	Active-low external reset input.

Upload a program

- Connect the board to the computer by USB and release the board reset button.
- Launch `python cpu_assembler_gui_fresh.py`; choose Nexys A7 mode.
- Click Refresh COM Ports, select the board COM port, set 921600 baud, and click Connect.
- Open your assembly program, include HLT, and click Send to FPGA or press F6.
- Wait for debug packets. The status panel and register table update as the CPU runs. When the CPU reaches HLT, a final packet reports halt=1.

Before uploading another program, press and release board reset so the loader can accept a new word stream. The loader expects words MSB byte first, then a final 0xFFFFFFFF marker. The GUI adds this marker automatically.

Debug packet (134 bytes)

Byte offsets	Contents
0	Header: 0xAA
1-4	Program counter (32-bit big-endian).
5-132	R0-R31 in order, each 4 bytes, big-endian.
133	Halt flag: 0 = running, 1 = halted.

The monitor sends packets periodically (about every 100 ms at the 100 MHz clock) and sends one final halted-state packet. “Program sent successfully” confirms the host finished writing bytes; use incoming packets and the HALTED state to confirm execution.

4. Architecture and troubleshooting

Memory / register reference

Resource	Implementation / usage
Registers	32 registers x 32 bits. R0 always reads as zero and ignores writes. Register file contents reset to zero.
Instruction memory	4096 x 32-bit words. The program counter is byte-addressed and increments by 4.
Data memory	1024 x 32-bit words. LW/SW addresses are byte addresses; RTL indexes by address / 4. Use aligned addresses.
CPU reset during load	CPU remains reset until the UART loader recognizes 0xFFFFFFFF as the end marker.
HLT	Source-level HLT instruction stops CPU execution; HLT is not the loader end marker.

Common issues

Symptom	What to check
GUI does not start	Run from the folder containing both Python files; verify Python and Tkinter.
No COM port appears	Check the USB cable, board power, driver, and click Refresh COM Ports.
Cannot connect to serial port	Close other terminal programs using the port. Confirm 921600 baud and 8-N-1.
Program transmitted but monitor is empty	Reset the board before a new upload; check selected COM port, programmed bitstream, and XDC UART pins.
Simulation cannot find files	Update D:/assembler references and the Vivado path in .bat/.tcl/testbench settings.
Simulation reaches timeout	Check that HLT is reachable, labels are correct, and branches do not loop forever.
LW/SW values are unexpected	Use aligned addresses and initialize data by storing it before loading.
Assembler rejects source	Check operand order, R0-R31 range, 16-bit immediate range, offset(Rs) syntax, and label spelling.

Implementation note. Data memory has no explicit reset initialization in the supplied RTL and no runtime bounds/alignment checks. Do not rely on an unwritten memory location having a predictable value. The archive also contains Vivado-generated folders and backup files; for a lean GitHub repository, keep the active RTL, project/XDC, testbench, scripts, and Python assembler/GUI, and include generated outputs only when useful to share.

For the detailed module inventory, instruction formats, and project notes, refer to the repository README.md.